

Disaster Recovery with Volsync

In Kubernetes environments, disaster recovery presents challenges, especially when storage systems lack native replication support. [VolSync](#) solves this problem by providing a Kubernetes operator that enables asynchronous replication of persistent volumes both within a cluster or across multiple clusters. This replication is independent of the underlying storage, making it compatible with various storage solutions.

What is Volsync?

VolSync facilitates volume replication across different storage systems in Kubernetes, even when the storage itself doesn't support remote replication. For instance, **Rook-Ceph** supports volume snapshots for its **Ceph RBD** and **CephFS** storage backends but lacks built-in volume replication. VolSync bridges this gap by enabling asynchronous replication across clusters or storage types.

Example VolSync Configuration

This section demonstrates how to configure disaster recovery for volumes using VolSync. Resources such as [PersistentVolumeClaims](#) (PVC), [ReplicationSources](#), and [ReplicationDestinations](#) are defined. This setup integrates with **Minio**, an S3-compatible storage solution, to manage backups and snapshots.

NOTE In this example, Flux performs `envsubst` to replace the variables in the YAML files.

Persistent Volume Claim (PVC)

The PVC is the core Kubernetes resource representing storage to be replicated. In VolSync, the PVC references a [ReplicationDestination](#) to restore data from a backup. When the PVC is created, data is automatically restored. If no backup exists, the PVC is created as an empty volume.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: "${APP}"
spec:
  accessModes: ["${VOLSYNC_ACCESSMODES:-ReadWriteOnce}"]
  dataSourceRef:
    kind: ReplicationDestination
    apiGroup: volsync.backube
    name: "${APP}-dst"
  resources:
    requests:
      storage: "${VOLSYNC_CAPACITY}"
```

```
storageClassName: "${VOLSYNC_STORAGECLASS:-ceph-block}"
```

In this example, Ceph is the storage backend, but replication can occur across different storage types.

External Secrets Configuration

VolSync stores backups in a remote repository like Minio. **ExternalSecrets** manages the credentials needed to access Minio securely.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: "${APP}-volsync"
spec:
  secretStoreRef:
    kind: ClusterSecretStore
    name: onepassword-connect
  target:
    name: "${APP}-volsync-secret"
    template:
      engineVersion: v2
      data:
        RESTIC_REPOSITORY: "s3:http://minio.minio-
system.svc.cluster.local:9000/volsync/${APP}"
        RESTIC_PASSWORD: "${ .RESTIC_PASSWORD }"
        AWS_ACCESS_KEY_ID: "${ .MINIO_ROOT_USER }"
        AWS_SECRET_ACCESS_KEY: "${ .MINIO_ROOT_PASSWORD }"
  dataFrom:
    - extract:
        key: minio
    - extract:
        key: volsync-minio-template
```

This configuration uses the ExternalSecrets operator to pull credentials from a secret store.

Replication Source

The **ReplicationSource** defines how and when data is backed up. It points to the PVC, schedules backups, and configures VolSync to use snapshots.

```
apiVersion: volsync.backube/v1alpha1
kind: ReplicationSource
metadata:
  name: "${APP}"
spec:
  sourcePVC: "${APP}"
```

```

trigger:
  schedule: "0 * * * *" // Backup every hour
restic:
  copyMethod: "${VOLSYNC_COPYMETHOD:-Snapshot}"
  pruneIntervalDays: 7
  repository: "${APP}-volsync-secret"
  volumeSnapshotClassName: "${VOLSYNC_SNAPSHOTCLASS:-csi-ceph-blockpool}"
  cacheCapacity: "${VOLSYNC_CACHE_CAPACITY:-4Gi}"
  cacheStorageClassName: "${VOLSYNC_CACHE_SNAPSHOTCLASS:-openebs-hostpath}"
  cacheAccessModes: ["${VOLSYNC_CACHE_ACCESSMODES:-ReadWriteOnce}"]
  storageClassName: "${VOLSYNC_STORAGECLASS:-ceph-block}"
  accessModes: ["${VOLSYNC_ACCESSMODES:-ReadWriteOnce}"]
  moverSecurityContext:
    runAsUser: 568
    runAsGroup: 568
    fsGroup: 568
  retain:
    hourly: 24
    daily: 7
    weekly: 5

```

This YAML defines how backups are triggered (in this case, every hour) and specifies where the backups are stored.

Replication Destination

The **ReplicationDestination** resource is used to restore data from a backup. It configures how to restore volumes, specifying parameters like storage class and access modes.

```

apiVersion: volsync.backube/v1alpha1
kind: ReplicationDestination
metadata:
  name: "${APP}-dst"
spec:
  trigger:
    manual: restore-once
  restic:
    repository: "${APP}-volsync-secret"
    copyMethod: Snapshot // must be Snapshot
    volumeSnapshotClassName: "${VOLSYNC_SNAPSHOTCLASS:-csi-ceph-blockpool}"
    cacheStorageClassName: "${VOLSYNC_CACHE_SNAPSHOTCLASS:-openebs-hostpath}"
    cacheAccessModes: ["${VOLSYNC_CACHE_ACCESSMODES:-ReadWriteOnce}"]
    cacheCapacity: "${VOLSYNC_CACHE_CAPACITY:-8Gi}"
    storageClassName: "${VOLSYNC_STORAGECLASS:-ceph-block}"
    accessModes: ["${VOLSYNC_ACCESSMODES:-ReadWriteOnce}"]
    capacity: "${VOLSYNC_CAPACITY}"

```

In this configuration, a manual restore operation is triggered, and snapshots are used to restore the

data.

NOTE

If no `latestImage` exists, the PVC will remain in a pending state until the `ReplicationDestination` completes a snapshot. Both the `ReplicationDestination` and the PVC referencing it can be created simultaneously. The PVC will start populating once a replication completes, and the snapshot (`latestImage`) becomes available. Additionally, if the storage class has a `volumeBindingMode` of `WaitForFirstConsumer`, the volume will only populate when a consumer (e.g., a pod) requests the PVC. At that time, VolSync will use the most recent snapshot from the `ReplicationDestination`.

Manual Operations for Managing VolSync Tasks

This section explains the manual operations defined in the Taskfile for managing VolSync in Kubernetes environments. These tasks automate various maintenance actions, including suspending controllers, listing snapshots, unlocking Restic repositories, creating snapshots, and restoring data for applications. Below are descriptions of the key tasks and their manual operations:

WARNING

Manual Operations: The following tasks are intended for manual execution and are not part of the automated VolSync workflow. A valid Kubernetes context is required to execute these tasks.

1. Suspend VolSync Controller

Task Name: `suspend-controller`

Purpose: This task suspends the VolSync operator and related Helm releases. It scales down the VolSync controller deployment to prevent any further replications or backups.

Commands:

- Suspend the VolSync operator and Helm release using Flux.
- Scale the VolSync controller deployment to 0 replicas.

Usage:

```
task volsync:suspend-controller --cluster <cluster-name>
```

2. List Snapshots for an Application

Task Name: `list`

Purpose: This task lists available Restic snapshots for a given application by creating a Kubernetes job to interact with the Restic repository.

Commands:

- Apply a job that lists snapshots using a template.
- Wait for the job to complete.
- Fetch logs from the job to display the list of snapshots.
- Clean up the job after completion.

Usage:

```
task volsync:list cluster=<cluster-name> app=<app-name> ns=<namespace>
```

3. Unlock a Restic Repository

Task Name: `unlock`

Purpose: Unlocks a Restic repository for an application. This is useful when locks are left behind after a failed backup or restoration operation.

Commands:

- Apply a job to unlock the Restic repository.
- Wait for the job to complete.
- Fetch logs to verify the unlock operation.
- Clean up the job after completion.

Usage:

```
task volsync:unlock cluster=<cluster-name> app=<app-name> ns=<namespace>
```

4. Create a Snapshot for an Application

Task Name: `snapshot`

Purpose: Triggers a manual snapshot for an application's `PVC` using the `ReplicationSource` resource

in Kubernetes.

Commands:

- Patch the **ReplicationSource** to trigger a manual snapshot.
- Wait for the snapshot process to complete.

Usage:

```
task volsync:snapshot cluster=<cluster-name> app=<app-name> ns=<namespace>
```

5. Restore a PVC for an Application

Task Name: **restore**

Purpose: Restores a **PVC** for an application from previous snapshots by using a combination of tasks that suspend the controller, wipe the **PVC**, restore data from the snapshots, and resume operations.

Commands:

- Suspend the Flux **kustomization** and Helm release.
- Wipe the current PVC data.
- Restore the data from a previous snapshot.
- Resume the Flux **kustomization** and Helm release.

Usage:

```
task volsync:restore cluster=<cluster-name> app=<app-name> ns=<namespace>  
previous=<num-snapshots>
```

6. Cleanup PVCs in All Namespaces

Task Name: **cleanup**

Purpose: This task deletes specific PVCs related to VolSync's volume population in all namespaces. It targets PVCs that match the criteria for destination or cache PVCs.

Commands:

- Iterate through and delete destination and cache PVCs across all namespaces.

Usage:

```
task volsync:cleanup cluster=<cluster-name>
```

Notes on Manual Operations

NOTE

Job Management: The manual tasks rely on Kubernetes Jobs to perform operations like listing snapshots, unlocking repositories, or wiping/restoring PVCs.